REPO INVESTIGATOR

RAG-Powered GitHub Code Intelligence

Detailed Project Report

Architecture • Tools • RAG Pipeline • MongoDB • Evaluation • Demo

Team ARISE — Team id 145421

Muntha Venkata Akash • B Sriram • Bodagala Venkata Ram Sharan • Mohith M • Kursam Girdhar

https://github.com/kursamgiridhar-web/rag_github_tool.git

Report Scope and Source Notes

This report is based primarily on the supplied rag_github_tool-main.zip source bundle, its README and demo documentation, and the supplied Team ARISE sample presentation. It describes implementation details and recorded results visible in those materials. Where the report describes a possible improvement, it is explicitly labelled as a proposal rather than an existing feature.

The supplied presentation shows a 20-question evaluation dashboard and before/after metrics. The README separately records an older 9/9 command-line retrieval evaluation. They are kept separate in this report because they represent different recorded contexts.

Contents

1. 1. Executive Summary
2. 2. Problem Statement and Goals
3. 3. Technology Stack and Tools Used
4. 4. System Architecture at a Glance
5. 5. Repository Ingestion Pipeline
6. 6. MongoDB Atlas: Data Model and Search
7. 7. Retrieval: Hybrid Search and Reciprocal Rank Fusion
8. 8. Answer Generation, Citations, and Reliability
9. 9. Frontend and User Experience
10. 10. Backend Modules and API
11. 11. Evaluation Methodology and Reported Results
12. 12. Security, Reliability, and Limitations
13. 13. Local Setup and Running the Project
14. 14. Recommended Hackathon Demo Script
15. 15. Hackathon Guideline Mapping
16. 16. Key Terms Glossary
17. 17. Conclusion

1. Executive Summary

Repo Investigator is a Retrieval-Augmented Generation (RAG) application designed to help developers understand unfamiliar public GitHub repositories. A user imports a repository, asks a natural-language question, and receives an explanation grounded in source-code evidence retrieved from relevant files. Numbered citations point to source paths and line ranges so users can inspect the evidence.

The core idea is to combine a language model with a searchable repository-specific knowledge base. Instead of asking the model to answer only from its general training, the application retrieves relevant code chunks from MongoDB Atlas and supplies those chunks to Gemini as evidence for answer generation.

- Imports public GitHub repositories through the GitHub API; it does not clone, install, or execute repository code.
- Splits eligible files into overlapping chunks while preserving source paths, line numbers, language, category, and architectural layer.
- Generates 768-dimensional embeddings with Gemini and stores them alongside chunk text and metadata in MongoDB Atlas.
- Combines MongoDB Atlas Vector Search and Atlas Search keyword retrieval, then fuses ranked results using Reciprocal Rank Fusion (RRF).
- Supports general questions, trace-style cross-file questions, and bug-investigation questions.
- Returns numbered source citations and a clear missing-evidence response when the retrieved context is insufficient.
- Includes a browser-based evaluation dashboard and a reusable evaluation question bank.

2. Problem Statement and Goals

Large software repositories distribute behavior across many files and layers. A developer may need to inspect a React component, API client, backend route, controller, database model, and configuration file to understand one feature. Conventional text search is useful for exact terms but often misses semantically related code or the connection between separate files.

Repo Investigator aims to reduce the time required to explore an unfamiliar codebase by making code searchable through natural-language questions and by presenting verifiable source evidence.

- Repository-wide exploration without manually opening every file.
- Cross-file reasoning about flows such as login, configuration, or database access.
- Hybrid retrieval that can find both conceptually similar passages and exact identifiers.
- Evidence-based explanations with file paths, source-line ranges, and clickable citations.
- Explicit handling of questions for which the repository does not contain sufficient evidence.

Note: Scope boundary: the application investigates and explains code. It does not automatically patch files, execute the imported project, or commit fixes.

3. Technology Stack and Tools Used

Layer / tool	Role in the project
React 18	Builds the interactive single-page user interface.
Vite	Runs the frontend development server and builds the frontend assets.
CSS	Styles the interface, panels, layouts, animations, and responsive behavior.
Node.js	Runs the JavaScript backend.
Express	Provides the HTTP API, routing, middleware, and error handling.
MongoDB Atlas	Persists repository records, code chunks, embeddings, evaluation runs, and searchable indexes.

MongoDB Atlas Vector Search	Retrieves semantically similar chunks using query embeddings.
MongoDB Atlas Search	Retrieves keyword matches, including identifiers and exact names.
Gemini API	Creates embeddings and generates evidence-grounded answers; also supports evaluation judging.
gemini-embedding-001	Configured embedding model; vectors use 768 dimensions.
GitHub REST API	Fetches public repository metadata, file trees, and file contents.
Three.js + React Three Fiber	Provides the 3D/network visual elements in the frontend.
Motion	Provides interface motion and transition effects.
Lucide React	Provides interface icons.
Prism React Renderer	Supports syntax-highlighted code display.
Browser localStorage	Stores saved conversations locally in the user's browser.
npm	Installs dependencies and runs the project scripts.

The exact configured default generation model is read from GEMINI_MODEL in the environment configuration, with fallback model names configured in code. Model availability and API quota can change; the report therefore distinguishes the configured model setting from the embedding model explicitly named by the code.

4. System Architecture at a Glance

The application has a React/Vite frontend, an Express backend, MongoDB Atlas for persistence and retrieval, the GitHub API for repository content, and Gemini for embeddings and answer generation.

18. The user submits a public GitHub repository URL through the frontend.
19. The backend validates/parses the URL and fetches repository metadata and eligible files.
20. The ingestion pipeline classifies files, chunks their content, creates embeddings, and stores chunk records in MongoDB Atlas.
21. MongoDB Atlas search indexes make the stored chunks available to vector and keyword retrieval.
22. The user submits a question and optional retrieval scope.
23. The backend identifies the question mode and retrieves relevant chunks using hybrid search.
24. RRF merges ranked results; mode-specific selection may add cross-layer evidence.
25. Gemini receives numbered evidence blocks and generates a structured, grounded response.
26. Citation references are checked against retrieved evidence and returned with source metadata.
27. The frontend displays the answer, citations, evidence panel, and source-code viewer.

> **Note:** The ingestion pipeline and question-answering pipeline are distinct. Ingestion prepares the repository for repeated searches; question answering retrieves a small, relevant subset of that stored evidence for each question.

5. Repository Ingestion Pipeline

5.1 Repository import

The user imports a public repository by URL. The backend uses GitHub API access to retrieve repository details, branch information, a recursive file tree, and file contents. A GitHub token is recommended to reduce unauthenticated API rate-limit issues. The code does not clone the repository or execute any of its code.

5.2 File filtering and classification

The classification stage filters files that are not useful for indexing, including binary/generated material and selected excluded files/directories. Eligible files receive metadata describing language, category, and architectural layer. The categories include code, configuration, tests, and documentation; layers include frontend, backend, database, and other.

5.3 Chunking

The chunking module is backend/src/chunking/chunker.js. It splits file content into overlapping sections, preserving one-based inclusive source line ranges. The configured defaults are up to 40 lines per chunk, up to about 2,000 characters per chunk, and an eight-line overlap. The overlap reduces the chance that a relevant function or explanation is lost at a chunk boundary.

5.4 Embedding

The embedding module calls Gemini's gemini-embedding-001 model. The configured vector dimension is 768. The text used for embedding includes chunk content and file-path context, helping retrieval account for both what the code says and where it lives. The implementation supports batched document embedding and retry behavior for transient failures or rate limits.

5.5 Storage and readiness

The indexRepo ingestion coordinator tracks progress through fetching, chunking, embedding, saving, and search preparation. Chunk documents and repository status are stored in MongoDB Atlas. Search indexes are prepared on backend startup and the application waits for search readiness before treating the repository as ready.

Default limit	Configured value	Purpose
Maximum files	120	Caps import size and keeps demos/API usage manageable.
Maximum bytes per file	80,000 bytes	Skips overly large files.
Maximum chunks per repository	500	Bounds storage, indexing time, and embedding usage.
Chunk line target	40 lines	Keeps retrieval units focused.
Chunk character target	About 2,000 characters	Prevents chunks from becoming too large.
Chunk overlap	8 lines	Preserves context across neighboring chunks.
Embedding dimensions	768	Matches the configured Atlas vector index dimensions.

6. MongoDB Atlas: Data Model and Search

MongoDB Atlas is both the persistent store and the retrieval substrate. The application stores searchable chunks with their vectors and metadata, rather than treating the database as a place to save only final chatbot responses.

6.1 Collection: repos

The repos collection stores repository-level records. Important fields include the repository URL, owner, name, branch, status, current indexing stage, progress, file count, chunk count, skipped-file information, error information, and created/updated timestamps. Fields can be populated differently depending on whether the repository is indexing, ready, or failed.

6.2 Collection: chunks

The chunks collection stores the individual retrievable evidence passages. Important fields include repoId, filePath, language, category, layer, startLine, endLine, text, embedding, and createdAt. The embedding is an array of numeric values, not a text string. filePath plus startLine/endLine supports source navigation and citation linking.

Field	Meaning
repoId	Associates the chunk with an imported repository.
filePath	Original path of the source file.
language	Detected language or file type.
category	Classification such as code, config, test, or docs.
layer	Architectural classification such as frontend, backend, database, or other.
startLine / endLine	One-based inclusive source line range represented by the chunk.

text	Original source text retained as retrievable evidence.
embedding	768-dimensional numeric vector for semantic retrieval.
createdAt	Timestamp associated with the chunk record.

6.3 Collection: eval_runs

The eval_runs collection stores evaluation-run state and results, including repository identity, status, start time, question count, progress, per-question results, retrieved file paths, metrics, and aggregate values. This allows the UI to poll a running evaluation and later inspect the recorded run.

6.4 Search indexes

- chunks_vector: Atlas Vector Search index on the embedding field, configured for cosine similarity and metadata filtering.
- chunks_text: Atlas Search index covering text, filePath, and repoId to support keyword-oriented retrieval.
- The exact index definitions are maintained in backend/src/db/mongo.js and are created/ensured by the backend startup logic.

6.5 What is not stored in MongoDB?

The README states that saved chat conversations are stored in the browser's localStorage, not in the MongoDB collections. The current data model does not describe a separate collection containing every complete original repository file; the indexed source content is stored as chunks with metadata and embeddings.

7. Retrieval: Hybrid Search and Reciprocal Rank Fusion

The retrieval implementation lives in backend/src/retrieval/search.js. It combines two complementary search approaches so that it can find both semantically relevant passages and exact textual matches.

7.1 Vector search

The question is embedded into a vector. Atlas Vector Search finds chunks whose embeddings are similar to the question embedding. This helps retrieve conceptually relevant content even when the question and code use different wording.

7.2 Keyword search

Atlas Search finds text matches, file-path matches, and identifiers. This is especially useful for exact symbols such as MONGO_URI, function names, filenames, or configuration keys.

7.3 Reciprocal Rank Fusion

The fuse() function combines the ranked lists. A simplified RRF formula is: score(d) = sum over retrieval lists of 1 / (k + rank(d)), with k commonly set to 60 in the project description. RRF uses the rank positions rather than assuming the vector and keyword raw scores are directly comparable.

7.4 Mode-aware evidence selection

- general: answers an ordinary repository question using relevant evidence.
- trace: seeks supporting chunks across frontend, backend, and database layers to explain a flow across files.
- bug: can supplement evidence with configuration and test-related context when investigating a possible issue.
- The retrieval code limits the final evidence set (up to 15 chunks) and uses a per-file cap to reduce one file from dominating the answer.
- Scope filtering can limit retrieval to all files, code, configuration, tests, or documentation.

The result is a compact evidence set sent to the answer-generation stage. RRF ranks and merges search results; it does not itself generate natural-language explanations.

8. Answer Generation, Citations, and Reliability

The answer-generation logic is in backend/src/llm/gemini.js. The backend supplies the question, the detected mode, and numbered evidence blocks containing source paths, line ranges, and retrieved text. Gemini is instructed to answer from the supplied evidence and return a structured response with answerability information.

28. Retrieve the relevant chunks and assign evidence numbers.
29. Construct a prompt that delimits repository content as untrusted data.
30. Ask Gemini to return a structured answer, including whether the evidence supports an answer.
31. Parse the model response and map citation numbers back to retrieved chunks.
32. Remove citation references that do not correspond to retrieved evidence.
33. Return the answer, answerable flag, mode, source records, and relevant metadata to the frontend.
34. If there is no sufficient supporting evidence, show a “Not found in repository” result rather than pretending the feature exists.

Answer modes

- General: a direct explanation supported by retrieved passages.
- Trace: a structured walkthrough across files/layers, such as a login request path.
- Bug: likely causes and checks grounded in code/configuration evidence. It should present possibilities rather than claim certainty without evidence.

Citation experience

The frontend shows numbered citation markers and an evidence panel. A user can inspect file path, line range, category, layer, and relevance information; open highlighted code in the code viewer; or navigate to the corresponding GitHub lines. Citation validation improves traceability, but it does not independently prove that every statement in the prose is correct.

Error handling

The backend has explicit HTTP error handling and avoids sending unexpected internal error details to clients. The README also notes retries for embedding requests and fallback generation models for certain quota/availability problems. GitHub rate limits and Gemini service errors can still affect the user experience.

9. Frontend and User Experience

The frontend is a React 18 single-page application built with Vite and plain CSS. It is designed around repository import, progress visibility, a chat/investigation workspace, source evidence, and evaluation.

File / component	Responsibility
App.jsx	Top-level application state and screen flow.
Welcome.jsx	Repository URL entry and recent repository entry points.
ConnectSequence.jsx	Shows indexing progress across the actual backend stages.
Shell.jsx	Main investigation workspace, sidebar, and navigation.
Composer.jsx	Question input and retrieval-scope controls.
Thread.jsx	Displays the conversation and generated answers.
EvidencePanel.jsx	Displays retrieved source evidence and citation details.
CodeViewer.jsx	Displays syntax-highlighted code and line navigation.
EvalDashboard.jsx	Displays evaluation runs, metrics, and question-level results.
api.js / evalApi.js	Frontend API client functions.
lib/history.js	Saves conversation history in browser localStorage.
lib/repoTree.js	Repository tree/file navigation helpers.
lib/smallTalk.js	Handles simple small-talk responses locally without retrieval.
styles/app.css	Main visual styling.

Visual libraries include Three.js and React Three Fiber for 3D/network backgrounds, Motion for animation, Lucide React for icons, and Prism React Renderer for syntax highlighting. The app also includes collapsible navigation and reduced-motion support.

10. Backend Modules and API

Module	Purpose
src/server.js	Starts Express, installs CORS/JSON middleware, mounts routes, handles errors, and initializes database/search readiness.
src/config.js	Reads environment variables and default limits.
src/ingestion/github.js	Parses repository URLs and fetches repository content through GitHub API.
src/ingestion/classify.js	Filters and classifies files; detects language, category, and layer.
src/chunking/chunker.js	Creates overlapping chunks and source line ranges.
src/embeddings/gemini.js	Creates document/query embeddings, normalizes vectors, batches requests, and retries transient failures.
src/ingestion/indexRepo.js	Orchestrates the import/index pipeline and progress updates.
src/db/mongo.js	Connects to MongoDB and ensures Atlas search indexes.
src/retrieval/search.js	Runs vector and keyword retrieval, RRF fusion, mode detection, and evidence selection.
src/llm/gemini.js	Builds grounded prompts, calls Gemini, parses answers, validates citations, and handles model fallbacks.
src/routes/repos.js	Repository import, listing, and status endpoints.
src/routes/ask.js	Question validation, retrieval/answer orchestration, and response construction.
src/routes/eval.js	Runs evaluation, records results, and serves dashboard data.
eval/questions.json	Evaluation questions, expected files, categories, and reference answers.
eval/run-eval.js	Command-line evaluation runner.

Principal API routes

Method / route	Purpose
GET /api/health	Backend health/configuration check.
POST /api/repos	Imports or reuses a ready repository; accepts repository URL and optional refresh behavior.
GET /api/repos	Lists indexed/ready repositories.
GET /api/repos/:id	Returns repository/indexing status.
POST /api/ask	Accepts repository ID, question, and scope; returns answerability, mode, sources, and metadata.
POST /api/eval/run	Starts an evaluation run; UI can poll progress.
GET /api/eval/questions	Returns the evaluation question set.
GET /api/eval/runs	Lists evaluation runs.
GET /api/eval/runs/:runId	Returns details for a particular run.

11. Evaluation Methodology and Reported Results

The repository includes backend/eval/questions.json and a command-line runner, as well as a backend evaluation API and frontend dashboard. The question set covers single-file, cross-file, frontend–backend–database, architecture, ambiguous, missing-file, unanswerable, multi-file reasoning, edge cases, incorrect assumptions, and incomplete questions.

Metrics

- Context recall: compares expected source-file paths with the files retrieved. Questions without expected supporting files are treated as not applicable for recall.

- Faithfulness: uses a Gemini-based judge to assess whether the generated answer is supported by retrieved evidence.
- Answer correctness: uses a Gemini-based judge to compare the generated answer with a reference answer.

The evaluation runner stores question-level data, retrieved files, chunks, status, and metrics in eval_runs, then calculates aggregate values. These measurements are useful for comparing iterations but depend on the test set, repository, judge behavior, model configuration, and API availability.

Results shown in the supplied team presentation

Metric	Before snapshot	After snapshot	Change
Context recall	91%	97%	+6 percentage points
Faithfulness	93%	94%	+1 percentage point
Answer correctness	86%	89%	+3 percentage points

The sample deck also shows a dashboard snapshot with 20 total questions, 20 passed, and 11 categories. Separately, the README records an older command-line result of 9/9 hit rate and mean recall 1.00 for the demo evaluation. These are different recorded contexts and must not be combined or described as one run.

> **Note:** Evaluation results are recorded snapshots, not guarantees that the system will achieve the same metrics on every repository or every future run.

12. Security, Reliability, and Limitations

Security and trust boundaries

- Only public GitHub repositories are supported in the documented setup.
- Repository code is never executed, installed, or run by the ingestion pipeline.
- Repository content is treated as untrusted input and delimited in the answer prompt so code comments or README instructions are not treated as privileged instructions.
- `.env` files are excluded from indexing.
- The backend hides unexpected internal error details from API clients.
- Citation references are checked against retrieved evidence; invalid references are removed.
- Environment files must never be committed or shared.

Operational limits

- Default limits: 120 files, 80,000 bytes per file, and 500 chunks per repository.
- Only public repositories are supported; private repository authorization is not a documented feature.
- GitHub unauthenticated access is more rate-limited; a token is recommended.
- Gemini free-tier quota/availability can delay ingestion or answer generation; retry/fallback behavior helps but cannot eliminate outages.
- Saved chats are browser-local and are not synchronized across devices; clearing site data can remove them.
- The README states that the project does not yet have a comprehensive automated software test suite beyond retrieval evaluation.
- Date-based and owner-based retrieval filters are not described as implemented user-facing filters; do not present them as current capabilities.

Future improvements (proposals, not current features)

- Private repository support with carefully scoped GitHub authentication.
- Persistent, account-based chat history synchronized across devices.
- More extensive automated unit, integration, and end-to-end tests.
- Expanded evaluation datasets and human review of model-judge scores.

- Optional date/owner metadata filters if the ingestion schema and query interface are extended.
- Incremental re-indexing and change detection for repositories that update frequently.

13. Local Setup and Running the Project

The README specifies Node.js 22 or later (tested on Node.js 24). The project uses npm scripts and separates frontend and backend dependencies.

Required services and credentials

- MongoDB Atlas cluster and database user, with network access configured for the development machine.
- Gemini API key from Google AI Studio.
- Optional/recommended GitHub personal access token for improved API rate limits.

Environment variables

Variable	Purpose
MONGODB_URI	MongoDB Atlas connection string.
MONGODB_DB	Database name; defaults to repo_investigator.
GEMINI_API_KEY	Gemini API credential.
GEMINI_MODEL	Primary answer-generation model setting.
GEMINI_FALLBACK_MODELS	Comma-separated fallback model names.
GEMINI_EMBEDDING_MODEL	Embedding model; defaults to gemini-embedding-001.
EMBEDDING_DIMENSIONS	Vector dimension; defaults to 768.
GITHUB_TOKEN	Optional GitHub API token.
PORT	Backend port; defaults to 5000.
MAX_FILES	Import file limit; defaults to 120.
MAX_FILE_BYTES	Per-file size limit; defaults to 80,000.
MAX_CHUNKS	Chunk limit; defaults to 500.
MIN_VECTOR_SCORE	Retrieval threshold configuration.

Install and run

```
cp .env.example backend/.env
# Fill in the required values in backend/.env
npm install
npm install --prefix backend
npm install --prefix frontend
npm run dev
```

By default, the backend runs on http://localhost:5000 and the Vite frontend on http://localhost:5173. The Vite development server proxies /api requests to the backend.

Run components separately

```
cd backend && npm run dev
# In a second terminal:
cd frontend && npm run dev
```

Build and evaluation commands

```
cd frontend && npm run build
cd backend && npm run eval
```

Note: Never commit backend/.env. Keep API keys and MongoDB connection strings private.

14. Hackathon Guideline Mapping

Guideline	How Repo Investigator addresses it
Import a collection and make it searchable	Imports a public GitHub repository, chunks its files, embeds the chunks, stores them in MongoDB Atlas, and prepares search indexes.
Ask a question requiring multiple sources	Trace mode can retrieve evidence across frontend, backend, and database layers.
Open citations and show exact passages	Numbered citations map to retrieved chunks with file paths and line ranges; the frontend evidence/code viewer exposes source passages.
Try weak/unanswerable request	The system can return a “Not found in repository” response when supporting evidence is missing.
Store source, chunks, embeddings, metadata, identifiers together	The chunks collection stores source text chunks, vectors, file paths, line ranges, category/layer metadata, and repository identifiers.
Use semantic retrieval	MongoDB Atlas Vector Search retrieves semantically similar chunks.
Use hybrid search for exact names/keywords	Atlas Search complements vector retrieval; RRF merges ranked results.
Use metadata filters	Implemented scope categories include all, code, config, tests, and docs; chunks also carry layer metadata.
Keep test requests and expected source IDs/files	backend/eval/questions.json stores questions and expected source files; the runner calculates recall and related metrics.

The guidelines mention date/owner filters as examples. The project should not claim these as implemented user-facing filters unless they are added and tested. The current implementation demonstrates metadata filtering through content scope and repository/layer metadata.

15. Key Terms Glossary

Term	Plain-language meaning
RAG	Retrieval-Augmented Generation: retrieve relevant information first, then use it as context for the model's answer.
Chunk	A small section of a source file stored as an independent retrieval unit.
Embedding	A numeric vector representing semantic features of text.
Vector Search	Search for vectors that are close in embedding space.
Keyword Search	Search based on words, identifiers, file paths, and textual matches.
Hybrid retrieval	Combining vector and keyword search.
RRF	Reciprocal Rank Fusion: a method for merging ranked search result lists.
Metadata	Structured information about a chunk, such as file path, language, category, layer, and line numbers.
Citation	A reference connecting an answer claim to a retrieved source chunk.
Context recall	How much of the expected source-file evidence was retrieved.
Faithfulness	How well the answer is supported by the retrieved evidence.
Answer correctness	How well the answer matches the reference answer or expected facts.
Indexing	Preparing repository content, chunks, vectors, and indexes so it can be searched efficiently.

16. Conclusion

Repo Investigator combines repository ingestion, source-aware chunking, Gemini embeddings, MongoDB Atlas hybrid search, RRF, evidence-grounded generation, citation validation, and evaluation in one developer-facing application. Its central value is not merely producing an AI response: it connects explanations to inspectable code evidence and gives an explicit missing-evidence response when the repository does not support an answer.

The most important points to communicate during the hackathon are the full ingestion-to-answer pipeline, the reason for hybrid retrieval, the MongoDB document structure, the cross-file evidence strategy, citation verification, the limitations of the current scope, and the evaluation methodology.

Appendix A — Repository and Setup Reference

Repository: https://github.com/kursamgiridhar-web/rag_github_tool.git

Backend: backend/src/ (server, configuration, ingestion, chunking, embeddings, database, retrieval, LLM, routes)

Frontend: frontend/src/ (React app, components, API clients, local history, repository tree utilities, styles)

Evaluation: backend/eval/questions.json and backend/eval/run-eval.js

Documentation: README.md, docs/DEMO.md, docs/IMPLEMENTATION_PLAN.md